Design a Web Crawler - Step 3: Design Deep Dive

1. Key Components in Deep Dive

The following core areas are analyzed in detail:

- DFS vs BFS
- URL Frontier
- HTML Downloader
- Robustness
- Extensibility
- Detecting problematic content

2. DFS vs BFS in Web Crawling

Web as a Graph

- Web can be modeled as a **directed graph**
    - Nodes → Web pages
    - Edges → Hyperlinks (URLs)

Crawling = Graph traversal

DFS (Depth-First Search)

- Traverses deeply into one path before exploring others

Problems with DFS

- Can go **very deep** → inefficient
- May get stuck in:
    - Infinite paths
    - Deep irrelevant pages
- Poor for large-scale crawling

Not suitable for web crawlers

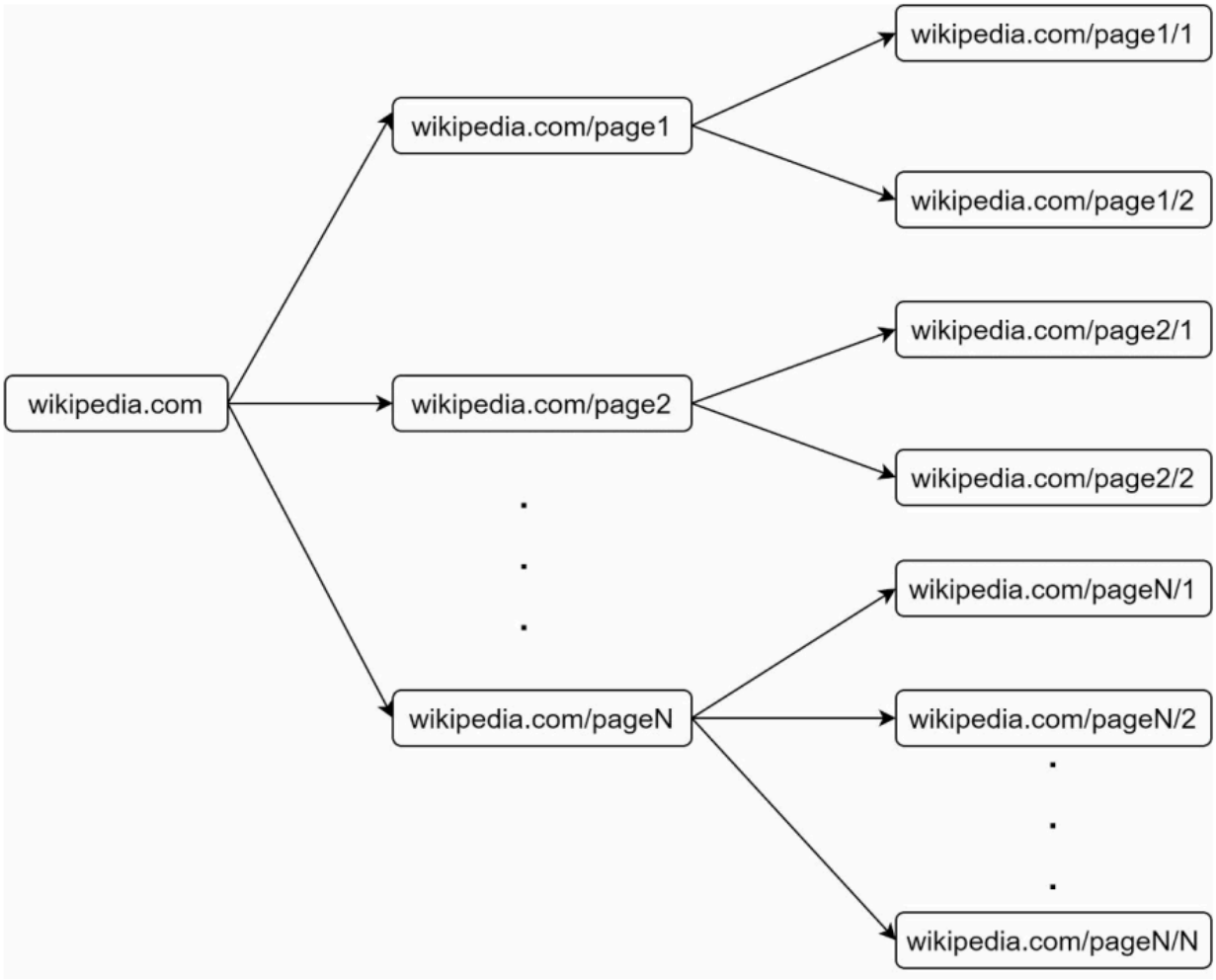
BFS (Breadth-First Search)

- Traverses level by level
- Implemented using **FIFO Queue**

Advantages

- Better coverage of web
- More balanced exploration
- Avoids deep traversal issues

Widely used in web crawlers



3. Issues with Standard BFS

Even though BFS is preferred, it has limitations:

Problem 1: Same Host Overloading

- Many links from a page point to the **same domain**

Example:

- wikipedia.com → mostly internal links

Issue:

- Crawler keeps hitting the same server
- Causes:
    - Server overload
    - Network congestion
    - Violates **politeness policy**

Problem 2: No URL Prioritization

- BFS treats all URLs equally

Issue:

- Important pages are not prioritized
- Low-quality pages consume resources

Why Prioritization is Needed

Some URLs are more valuable based on:

- Page rank
- Traffic
- Update frequency
- Domain importance

Without prioritization → inefficient crawling

4. Key Takeaway

BFS is preferred over DFS because:

- Better scalability
- Balanced traversal

But needs improvements:

- Host-based scheduling (avoid overload)
- Priority-based crawling